

📖 Material Java — Modificadores de Acesso, Construtores e Boas Práticas

Guia didático sobre encapsulamento (`public`, `private`, `protected`, `default`), sobrecarga de construtores e boas práticas de POO. Continuação da série de estudos, seguindo a metodologia DevSuperior.

Sumário

1. O que são modificadores de acesso
2. `public`
3. `private`
4. `protected`
5. `default` (sem modificador)
6. Tabela comparativa de visibilidade
7. Encapsulamento: `private` + getters/setters
8. Sobrecarga de construtores
9. Boas práticas

1. O que são modificadores de acesso

São palavras que controlam **quem pode ver e usar** um atributo, método ou classe. Definem a "visibilidade" de cada parte do código.

A ideia central é o **encapsulamento**: esconder os detalhes internos de uma classe e expor só o necessário. Como um carro — você usa o volante e os pedais (interface pública), sem precisar mexer no motor (implementação interna, protegida).

São quatro níveis, do mais aberto ao mais fechado:

```
public → protected → default → private
(todos)                               (só a própria classe)
```

Aplicam-se a três coisas: - **Atributos** (variáveis da classe) - **Métodos** - **A própria classe**

2. `public`

Visível para TODOS — qualquer classe, de qualquer pacote, pode acessar.

```
public class Produto {
    public String nome;           // qualquer código pode ler/alterar
    public double calcularTotal() { ... } // qualquer código pode chamar
}
```

Quando usar: - Métodos que formam a "interface" da classe — o que ela oferece ao mundo externo - Construtores (geralmente públicos, para permitir criar objetos de qualquer lugar) - A classe principal de um arquivo

Cuidado: atributos `public` são geralmente **evitados** — expor os dados diretamente quebra o encapsulamento (qualquer um pode alterá-los sem controle). Prefira `private` + getters/setters (seção 7).

3. private

Visível SOMENTE dentro da própria classe. Nenhuma outra classe consegue acessar — nem subclasses, nem classes do mesmo pacote.

```
public class ContaBancaria {
    private double saldo; // só a própria classe mexe no saldo

    public void depositar(double valor) {
        if (valor > 0) {
            saldo += valor; // acesso permitido: está DENTRO da classe
        }
    }
}
```

Tentar `conta.saldo = 1000000;` de fora → **erro de compilação**. O saldo só muda pelos métodos que a classe oferece (`depositar`, `sacar`), que podem validar as regras.

Quando usar: - **Atributos** — quase sempre `private`, para proteger os dados. Este é o padrão do encapsulamento. - Métodos auxiliares internos que só interessam à própria classe

💡 `private` é o modificador mais usado para atributos. Ele garante que os dados só sejam alterados de forma controlada, através dos métodos da classe.

4. protected

Visível na própria classe, no mesmo pacote, e nas subclasses (herança) — mesmo que a subclasse esteja em outro pacote.

```
public class Animal {
    protected String especie; // acessível por subclasses de Animal
}

public class Cachorro extends Animal {
    void exibir() {
        System.out.println(especie); // OK — herdou de Animal
    }
}
```

Quando usar: - Quando você trabalha com **herança** e quer que as subclasses acessem certos membros da classe-pai, mas não o resto do mundo.

⚠ `protected` só faz sentido com **herança** (`extends`), tema que vem depois de encapsulamento. Por ora, basta saber que ele existe e é "entre o `private` e o `public`" — mais aberto que `private` (inclui subclasses), mais fechado que `public` (não é todo mundo).

5. default (sem modificador)

Quando você **não escreve** modificador nenhum, o acesso é o "default" (também chamado *package-private*): **visível apenas dentro do mesmo pacote**.

```
class Utilitario {          // sem 'public' → visível só no mesmo pacote
    int contador;           // sem modificador → default
    void ajudar() { ... }   // default
}
```

Classes de outros pacotes **não** enxergam membros default.

Quando usar: - Classes/métodos auxiliares que só interessam dentro de um pacote específico - É comum em código de estudo (você vem usando default sem perceber — os atributos do seu `Produto` sem `private`/`public` são default)

💡 **Nota:** nos seus exercícios até agora, atributos como `String nome;` (sem modificador) são **default**. Funcionam porque tudo está no mesmo pacote. Ao evoluir para encapsulamento, esses viram `private`.

6. Tabela comparativa de visibilidade

Quem consegue acessar cada nível:

Modificador	Mesma classe	Mesmo pacote	Subclasse (outro pacote)	Qualquer lugar
<code>public</code>	✓	✓	✓	✓
<code>protected</code>	✓	✓	✓	✗
<code>default</code>	✓	✓	✗	✗
<code>private</code>	✓	✗	✗	✗

Do mais aberto (`public`, todos) ao mais fechado (`private`, só a própria classe). O `default` é o "meio-termo silencioso" (sem palavra-chave), e o `protected` adiciona as subclasses ao alcance do pacote.

7. Encapsulamento: private + getters/setters

O padrão profissional para atributos: deixá-los `private` e controlar o acesso por métodos públicos chamados **getters** (ler) e **setters** (alterar).

```
public class Pessoa {
    private String nome;    // protegido
    private int idade;

    // GETTER — permite LER o valor
    public String getNome() {
        return nome;
    }

    // SETTER — permite ALTERAR, com validação
    public void setIdade(int idade) {
        if (idade >= 0) {    // regra: idade não pode ser negativa
            this.idade = idade;
        } else {
            System.out.println("Idade inválida.");
        }
    }

    public int getIdade() {
        return idade;
    }
}
```

Por que isso é melhor que atributos públicos

Com atributo `public`, qualquer um faz `pessoa.idade = -50;` — um estado inválido, sem controle. Com `private` + setter, a classe **valida** antes de aceitar: idade negativa é rejeitada. Os dados ficam sempre consistentes.

Benefícios: - **Controle** — o setter valida antes de alterar (regras de negócio protegidas) - **Flexibilidade** — você pode mudar a implementação interna sem afetar quem usa a classe - **Somente-leitura** — dá para criar só o getter (sem setter), tornando o atributo imutável de fora

Atalho no IntelliJ

`Alt + Insert` → **Getter and Setter** → seleciona os atributos → gera tudo automaticamente. (Mesmo menu Generate do construtor.)

8. Sobrecarga de construtores

Sobrecarga (*overloading*) é ter **vários construtores** na mesma classe, cada um com uma **lista de parâmetros diferente**. O Java escolhe qual usar pela quantidade/tipo de argumentos que você passa no `new`.

```
public class Produto {
    private String nome;
    private double preco;
    private int estoque;

    // Construtor 1 – completo
    public Produto(String nome, double preco, int estoque) {
        this.nome = nome;
        this.preco = preco;
        this.estoque = estoque;
    }

    // Construtor 2 – sem estoque (assume 0)
    public Produto(String nome, double preco) {
        this.nome = nome;
        this.preco = preco;
        this.estoque = 0;    // valor padrão
    }

    // Construtor 3 – vazio (tudo padrão)
    public Produto() {
        this.nome = "Sem nome";
        this.preco = 0.0;
        this.estoque = 0;
    }
}
```

Uso — o Java escolhe pelo número de argumentos:

```
Produto p1 = new Produto("Mouse", 80.0, 50); // usa construtor 1
Produto p2 = new Produto("Mouse", 80.0);      // usa construtor 2 (estoque = 0)
Produto p3 = new Produto();                   // usa construtor 3 (tudo padrão)
```

A regra da sobrecarga

Os construtores precisam diferir na **assinatura** — quantidade ou tipo de parâmetros. O Java distingue qual chamar por isso. Não pode haver dois construtores com exatamente a mesma lista de parâmetros (seria ambíguo).

`this()` — **um construtor chamando outro**

Para evitar repetir código, um construtor pode chamar outro com `this(...)`:

```
public Produto(String nome, double preco) {
    this(nome, preco, 0); // chama o construtor completo com estoque = 0
}
```

Isso reaproveita a lógica do construtor principal — princípio DRY (não repetir código). `this(...)` deve ser a **primeira** linha do construtor.

Para que serve na prática

- **Flexibilidade** — criar objetos com diferentes níveis de informação
- **Valores padrão** — quando alguns dados não são fornecidos, o construtor assume padrões
- Frameworks (como Spring) frequentemente exigem um **construtor vazio** além do completo

9. Boas práticas

Encapsulamento

1. **Atributos sempre `private`** por padrão. Exponha via getters/setters conforme a necessidade — não crie setters "automáticos" para tudo; só onde alterar faz sentido.
2. **Getters/setters não são obrigatórios para todo atributo.** Se um dado não deve ser alterado de fora, crie só o getter (ou nenhum).
3. **Valide dentro do setter/construtor.** Idade negativa, preço negativo, string vazia — rejeite estados inválidos na entrada.

Construtores

1. **Construtor inicializa, não faz I/O.** Nada de `System.out.println` ou `Scanner` dentro do construtor — a responsabilidade dele é preparar o objeto, não interagir com o usuário. (Efeitos colaterais no construtor surpreendem quem cria o objeto.)
2. **Garanta objeto em estado válido.** O construtor deve deixar o objeto pronto para uso — todos os atributos essenciais inicializados.
3. **Use `this(...)` para evitar duplicação** entre construtores sobrecarregados.

Separação de responsabilidades

1. **Classe de entidade não lê teclado.** Uma classe que representa algo (Produto, Aluno, Conta) guarda dados e comportamentos; a leitura de entrada (Scanner) fica em outra classe (Main, Menu). Assim a entidade funciona independente da origem dos dados (teclado, arquivo, banco, API).
2. **Uma classe, uma responsabilidade.** Se uma classe faz coisas demais (representa dados E lê teclado E gera relatório), considere dividir.

Convenções de nome

1. **Classes** em PascalCase (`ContaBancaria`), **métodos e atributos** em camelCase (`calcularTotal`, `saldoAtual`), **constantes** em MAIÚSCULAS (`TAXA_JUROS`).
2. **Getters/setters seguem o padrão** `getNome()` / `setNome(...)`; para boolean, `isAtivo()`.

Visibilidade

1. **Comece fechado, abra conforme a necessidade.** Prefira `private`; só torne `public` o que realmente precisa ser acessado de fora. É mais fácil abrir depois do que fechar (algo público pode já estar sendo usado por outros).

Resumo em uma frase

Encapsular = esconder os dados (`private`) e expor comportamento controlado (métodos `public`). **Sobrecarregar construtores** = oferecer várias formas de criar o objeto. **Boa prática** = objeto nasce válido, se protege de estados inválidos, e cada classe cuida de uma responsabilidade.

Material em evolução — atualizado conforme o avanço nos estudos.

Jader Silva · GitHub: MONTANHA702 · LinkedIn: /in/jaderssantos/